

Label Leakage in Code Vulnerability Datasets: Evidence from TF-IDF Feature Analysis

Turing
RTSS Board
turingrtss@gmail.com

September 2026

Abstract

Machine learning approaches to vulnerability detection report high accuracy on benchmark datasets, but the source of this accuracy is rarely interrogated. We train TF-IDF classifiers on a publicly available code vulnerability dataset (8,480 samples, 14 classes) and achieve 84.7% binary classification accuracy. Feature coefficient analysis reveals that the model’s strongest predictive signals are character n-gram substrings of the words “vulnerable” and “safe,” which appear in code comments and variable names rather than in executable code patterns. This label leakage inflates reported accuracy and raises questions about the validity of benchmarks constructed from code samples containing natural language descriptions of their own labels. We quantify the leakage, identify which features correspond to genuine code patterns versus label artifacts, and propose dataset construction guidelines to mitigate the problem.

1 Introduction

The application of machine learning to automated vulnerability detection has produced a growing body of literature reporting classification accuracies above 80% on benchmark datasets [1, 2]. These results have motivated the development of ML-based static analysis tools intended to complement or replace traditional pattern-matching approaches such as regular expression scanners.

However, high accuracy on a benchmark does not guarantee that a model has learned meaningful vulnerability patterns. If the training data contains artifacts that correlate with labels but do not generalize to unseen code, the reported performance is misleading. One such artifact is *label leakage*: the presence of label-correlated information in input features that would not exist in a deployment setting.

In natural language processing, label leakage through metadata, formatting, or annotation artifacts is a well-documented failure mode [3]. In code vulnerability datasets, an analogous risk exists: code samples may contain comments, docstrings, or variable names that describe the vulnerability type, effectively encoding the classification label in the input.

This paper makes three contributions:

1. We demonstrate that a simple TF-IDF baseline achieves accuracy competitive with more complex models on a public vulnerability dataset, suggesting the task may be easier than it appears.
2. We identify specific label leakage through feature coefficient analysis, showing that the model’s top predictive features are substrings of label-descriptive natural language rather than code constructs.
3. We propose dataset construction guidelines to mitigate label leakage in future vulnerability detection benchmarks.

2 Methods

2.1 Dataset

We use the `Code_Vulnerability_Labeled_Dataset` [4] from HuggingFace, containing 8,480 code samples across multiple programming languages. Each sample consists of a code snippet and a label. Labels include “safe” (4,461 samples, 52.6%) and 13 CWE vulnerability types (4,019 samples, 47.4%). The distribution is shown in Table 1.

Table 1: Dataset distribution by label.

Label	Count
Safe	4,461
Out-of-bounds Write (CWE-787)	1,173
Code Injection (CWE-94)	781
Improper Input Validation (CWE-20)	631
NULL Pointer Dereference (CWE-476)	433
Deserialization (CWE-502)	301
SQL Injection (CWE-89)	241
Cross-site Scripting (CWE-79)	230
Integer Overflow (CWE-190)	103
XXE (CWE-611)	51
Log Injection (CWE-117)	33
OS Command Injection (CWE-78)	19
Open Redirect (CWE-601)	13
Path Traversal (CWE-22)	10

The dataset was not filtered by programming language. No preprocessing (comment stripping, identifier normalization) was applied, as the purpose of this study is to evaluate the dataset as-is.

2.2 Feature Extraction

Code samples were vectorized using TF-IDF with character-level n-grams (window size 3–6), sublinear term frequency scaling, and a maximum vocabulary of 50,000 features. Character n-grams were chosen over word-level tokens because code structure is better captured at the sub-token level: operators, delimiters, and function call patterns often span fewer than one whitespace-delimited word.

2.3 Models

Three classifiers were evaluated:

1. **Logistic Regression** (LR): $C = 1.0$, L2 regularization, max 1,000 iterations. Chosen as a linear baseline whose coefficients are directly interpretable.
2. **Random Forest** (RF): 100 estimators, parallelized across 2 cores.
3. **Gradient Boosting** (GB): 100 estimators, default learning rate.

2.4 Evaluation

All experiments used a stratified 80/20 train/test split with a fixed random seed (42). We report accuracy, precision, recall, and F1-score for the vulnerable class in binary classification, and per-

class F1 in multi-class classification. False positive rate (FPR) is computed as the proportion of safe samples incorrectly classified as vulnerable.

For label leakage analysis, we extract the logistic regression coefficient vector and rank features by their signed contribution to the vulnerable class prediction.

2.5 Hardware

All experiments were conducted on a single ARM64 server: Oracle Cloud free-tier, 2-core Neoverse-N1, 12 GB RAM, no GPU, Ubuntu 22.04.

3 Results

3.1 Binary Classification

Table 2 summarizes binary classification performance across all three models.

Table 2: Binary classification: vulnerable vs. safe.

Model	Acc.	Prec.	Rec.	F1	FPR
Logistic Regression	0.847	0.823	0.862	0.842	16.7%
Random Forest	0.827	0.808	0.833	0.821	17.8%
Gradient Boosting	0.837	0.801	0.872	0.835	19.5%

Logistic regression achieved the highest accuracy (0.847) and lowest false positive rate (16.7%) while training in 0.5 seconds. The nonlinear models offered no improvement, with gradient boosting requiring $350\times$ longer to train for a lower F1 score.

3.2 Multi-class Classification

Table 3 shows per-class F1 scores for 14-class logistic regression (overall accuracy: 0.725).

Table 3: Multi-class F1 scores by vulnerability type.

Type	F1	Support
Safe	0.82	892
Deserialization (CWE-502)	0.77	60
OOB Write (CWE-787)	0.65	235
NULL Pointer (CWE-476)	0.65	86
Code Injection (CWE-94)	0.60	156
Input Validation (CWE-20)	0.55	126
XSS (CWE-79)	0.41	46
SQL Injection (CWE-89)	0.34	48
Integer Overflow (CWE-190)	0.09	21
Path Traversal (CWE-22)	0.00	2
Cmd Injection (CWE-78)	0.00	4
Log Injection (CWE-117)	0.00	7
XXE (CWE-611)	0.00	10
Open Redirect (CWE-601)	0.00	3

All classes with fewer than 20 test samples achieved F1 of 0.00, indicating that the model cannot generalize from extremely small class sizes.

3.3 Feature Analysis: Label Leakage

Tables 4 and 5 show the top 10 logistic regression features for each class, ranked by coefficient magnitude.

Table 4: Top 10 features predicting the vulnerable class.

Feature (n-gram)	Coef.	Type
' this '	+1.412	NL
' eval('	+1.332	Code
' + '	+1.298	Ambiguous
' vuln'	+1.227	NL (label)
' vu'	+1.227	NL (label)
' vul'	+1.227	NL (label)
' thi'	+1.216	NL
' ev'	+1.206	Ambiguous
'vul'	+1.204	NL (label)
'uln'	+1.204	NL (label)

Table 5: Top 10 features predicting the safe class.

Feature (n-gram)	Coef.	Type
' sa'	−2.057	NL (label)
' saf'	−1.807	NL (label)
' safe'	−1.807	NL (label)
' if'	−1.779	Code
'secu'	−1.779	NL
'secur'	−1.779	NL
'ure'	−1.778	NL
' ?:'	−1.773	Code
'cur'	−1.766	NL
'ecur'	−1.746	NL

We manually categorized each feature as *Code* (executable syntax), *NL* (natural language in comments/names), *NL (label)* (substring of the classification label itself), or *Ambiguous*. Of the top 20 features across both classes:

- **8 features (40%)** are direct substrings of classification labels (“vuln,” “vul,” “uln,” “sa,” “saf,” “safe”).
- **6 features (30%)** are natural language tokens unrelated to code syntax (“this,” “secu,” “secur”).
- **4 features (20%)** correspond to code constructs (`eval()`, `if`, `?:`).
- **2 features (10%)** are ambiguous (+, `ev`).

4 Discussion

4.1 Interpretation of Results

The feature analysis demonstrates that 70% of the model’s top predictive signals derive from natural language rather than code structure. The single strongest vulnerability predictor across the entire feature space is a substring of the word “vulnerable,” which appears in comments and variable names within the dataset. This constitutes label leakage: the classification target is partially encoded in the input through descriptive annotations that would not be present in production code.

The dominance of the linear model is consistent with this interpretation. If the classification boundary is primarily defined by the presence or absence of label-descriptive words in comments, a linear classifier in TF-IDF space is sufficient to capture it. Nonlinear models cannot improve on this because the additional capacity they offer—modeling complex code interactions—is not rewarded by the data.

4.2 Implications for Benchmark Validity

These results have implications beyond this specific dataset. Any code vulnerability dataset constructed from samples that include descriptive comments, educational annotations, or label-derived variable names (e.g., `vulnerable_function`) risks the same leakage. Models trained on such data will exhibit inflated accuracy that does not transfer to unannotated production code.

The 84.7% accuracy reported here should not be interpreted as evidence that TF-IDF classifiers can detect vulnerabilities with 84.7% reliability. Rather, it reflects a mixture of genuine pattern recognition (e.g., detecting `eval()`) and shortcut learning through label-correlated text.

4.3 Quantifying the Leakage

While a precise decomposition of accuracy into “genuine” and “leaked” components would require a controlled ablation study (training on comment-stripped data and comparing), the feature analysis provides an upper bound on the contribution of code-related features. With only 20% of the top predictive features corresponding to executable code patterns, the genuine vulnerability detection capability of the model is likely substantially below the reported 84.7%.

4.4 Limitations

This study has several limitations:

1. We evaluate only one dataset. Other vulnerability datasets (e.g., DiverseVul [1], SecurityEval [2]) may exhibit different degrees of leakage.
2. We do not perform the ablation study (comment stripping + retraining) that would precisely quantify the accuracy drop. This is planned as future work.
3. TF-IDF with character n-grams is a simple featurization. Transformer-based models operating on abstract syntax trees might learn genuine code patterns even in the presence of comment-based leakage, though they would also likely exploit the leakage.
4. The feature categorization (Code vs. NL vs. NL-label) was performed manually and may contain subjective judgments.

4.5 Recommendations

Based on these findings, we propose the following guidelines for constructing code vulnerability datasets:

1. **Strip comments and docstrings** from all samples before release. This eliminates the primary leakage vector.
2. **Normalize identifiers** by replacing variable and function names with generic tokens (`var_1`, `func_2`). This prevents the model from learning naming conventions that correlate with labels.
3. **Source from CVE fix commits.** Before/after code pairs from real vulnerability patches provide naturally clean labels without descriptive annotations. The “before” version is vulnerable; the “after” version is the fix.
4. **Report feature analysis alongside accuracy.** Any benchmark paper should include an examination of top predictive features to demonstrate that the model is learning code patterns rather than annotation artifacts.

5 Conclusion

We trained TF-IDF classifiers on a public code vulnerability dataset and achieved 84.7% accuracy. Feature analysis revealed that 70% of the top predictive features are natural language artifacts—including direct substrings of classification labels—rather than code patterns. This label leakage inflates reported accuracy and calls into question the validity of benchmarks that do not control for it. We recommend comment stripping, identifier normalization, and mandatory feature analysis as standard practices for vulnerability detection dataset construction and evaluation.

Data and Code Availability

All code, data references, and experimental results are available at <https://github.com/turingrtss/vulndetect>. The dataset is publicly hosted on HuggingFace as `lemon42-ai/Code_Vulnerability_Labeled_Dataset`. Training completes in under 4 minutes on a 2-core ARM CPU with no GPU.

References

- [1] Y. Chen, Z. Ding, X. Chen, and D. Lo, “DiverseVul: A new vulnerable source code dataset for deep learning based vulnerability detection,” in *Proc. 20th Int. Conf. Mining Softw. Repositories (MSR)*, Melbourne, Australia, 2023, pp. 654–668.
- [2] M. A. Rahman, M. R. Parvez, S. Chakraborty, and M. A. Hasan, “SecurityEval dataset: Mining vulnerability examples to evaluate machine learning-based code generation techniques,” in *Proc. 1st Int. Workshop Mining Softw. Repositories Privacy Security (MSR4P&S)*, Singapore, 2022, pp. 17–21.
- [3] E. M. Bender and A. Koller, “Climbing towards NLU: On meaning, form, and understanding in the age of data,” in *Proc. 58th Annu. Meeting Assoc. Comput. Linguistics (ACL)*, 2020, pp. 5185–5198.
- [4] lemon42-ai, “Code_Vulnerability_Labeled_Dataset,” HuggingFace Datasets, 2024. [Online]. Available: https://huggingface.co/datasets/lemon42-ai/Code_Vulnerability_Labeled_Dataset